

Linear Programming Course Summary

Generated by Scribe.AI

December 4, 2024

1 Key Terms

- **Slack Variable:** A variable added to a less-than-or-equal-to inequality constraint to transform it into an equality constraint. It represents the difference between the left-hand side and right-hand side of the inequality.
- **Feasible Region:** The set of all points that satisfy all the constraints of a linear programming problem.
- **Vertex:** A point where two or more constraint boundaries intersect in the feasible region of a linear programming problem. In the Simplex method, optimal solutions are found at vertices.
- **Basic Variable:** In the Simplex method, a variable that is expressed in terms of non-basic variables in a dictionary. In a basic feasible solution, basic variables are non-negative.
- **Non-basic Variable:** In the Simplex method, a variable set to zero in a given iteration of the algorithm. The values of the basic variables are determined by the values of the non-basic variables.
- **Pivot Operation:** The process of exchanging a basic and a non-basic variable in the Simplex method, leading to a new basic feasible solution.
- **Dictionary:** A representation of a linear programming problem where basic variables are expressed as functions of non-basic variables. It is used in the Simplex method to systematically move from one vertex to another.
- **Auxiliary Problem:** A modified linear programming problem introduced to find an initial feasible solution when the origin is not feasible in the original problem. It involves adding a new variable to shift the feasible region.
- **Largest-Coefficient Rule:** A pivot rule in the Simplex method that selects the entering variable with the largest coefficient in the objective function row of the dictionary.
- **Largest-Increase Rule:** A pivot rule in the Simplex method that selects the entering variable that results in the largest increase in the objective function value.
- **Klee-Minty Example:** A worst-case example for the Simplex method, demonstrating that the number of iterations can grow exponentially with the problem size.
- **Bland's Rule:** A pivot rule in the Simplex method designed to prevent cycling, ensuring termination.
- **Lexicographic Method:** A method for choosing the leaving variable in the Simplex method that ensures termination by preventing cycling.
- **Randomized Pivot Rule:** A pivot rule that randomly permutes the order of variables before applying another rule, such as Bland's rule.
- **Smoothed Complexity:** A framework for analyzing the complexity of algorithms by considering the average-case performance after small random perturbations to the input.

- **Interior Method:** A class of algorithms for linear programming that traverse the interior of the feasible region, in contrast to the Simplex method which follows the boundary.
- **Primal Problem:** A linear programming problem that aims to maximize an objective function subject to a set of constraints and non-negativity conditions.
- **Dual Problem:** A linear programming problem derived from the primal problem, aiming to minimize an objective function subject to a different set of constraints and non-negativity conditions. The optimal solution of the dual problem provides valuable information about the primal problem.
- **Weak Duality:** A theorem stating that the objective function value of any feasible solution to the primal problem is always less than or equal to the objective function value of any feasible solution to the dual problem.
- **Negative Transpose Property:** The property that the coefficient matrix of the dual problem is the negative transpose of the coefficient matrix of the primal problem. This property is preserved throughout the simplex method iterations.
- **Strong Duality Theorem:** A theorem stating that if a linear programming problem has an optimal solution, then its dual problem also has an optimal solution, and the optimal objective function values of both problems are equal.
- **Complementary Slackness:** A condition stating that for optimal primal and dual solutions, the product of each primal variable and its corresponding dual slack variable is zero, and the product of each dual variable and its corresponding primal slack variable is zero.
- **Lagrange Multiplier:** A scalar value associated with each constraint in a constrained optimization problem, used to incorporate the constraints into the objective function via the Lagrangian.
- **Lagrangian Function:** A function that combines the objective function and constraints of a constrained optimization problem using Lagrange multipliers. The Lagrangian is unconstrained in the original variables.
- **Weak Duality Theorem:** A theorem stating that the optimal value of the primal problem is always less than or equal to the optimal value of the dual problem.
- **Diet Problem:** A linear programming problem where the objective is to minimize the cost of a diet while meeting minimum nutritional requirements.
- **Nutrient:** A substance that provides nourishment essential for growth and the maintenance of life.
- **Food:** An edible substance that provides nutrients.
- **Constraint Matrix:** A matrix representing the coefficients of the variables in the constraints of a linear programming problem.
- **Decision Variables:** Variables representing the quantities to be determined in an optimization problem.
- **Objective Function:** The function to be minimized or maximized in an optimization problem.
- **\textbackslash{}textbf\textbackslash{}{}Standard Form (Linear Programming)\textbackslash{}{}:**
A linear programming problem expressed as maximizing $\mathbf{c}^T \mathbf{x}$ subject to $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq \mathbf{0}$.
- **\textbackslash{}textbf\textbackslash{}{}Augmented Matrix\textbackslash{}{}:** A matrix formed by combining the constraint matrix $\mathbf{A}$ with the identity matrix $\mathbf{I}$ to represent the constraints in a linear programming problem after introducing slack variables.
- **\textbackslash{}textbf\textbackslash{}{}Simplex Iterates\textbackslash{}{}:** The sequence of solutions generated by the simplex method during its iterative process of improving the objective function value.

- **Basic Variables** ($\mathbf{x}$): A subset of variables in a linear programming problem that are set to non-zero values to determine a basic feasible solution. The number of basic variables equals the number of constraints.
- **Non-Basic Variables** ($\mathbf{z}$): The remaining variables in a linear programming problem that are set to zero to determine a basic feasible solution.
- **Simplex Tableau**: A tabular representation of the simplex method's matrix operations, showing the current values of the variables, objective function, and coefficients needed for updating the solution during iterations.
- **Pivot Operation**: The process of selecting a pivot element in the simplex tableau and performing row operations to update the tableau and move to the next iteration.
- **Complementary Slackness**: A condition that relates the primal and dual variables in a linear programming problem, stating that at optimality, either a primal variable or its corresponding dual variable must be zero.
- **Optimal Solution**: The solution to a linear programming problem that maximizes (or minimizes) the objective function while satisfying all constraints.
- **Shadow Price**: The rate of change in the optimal objective function value for a unit change in the right-hand side of a constraint.
- **Reduced Costs**: The vector $z_N = (B^{-1}N)^T c_B - c_N$, representing the change in the objective function value per unit increase in a non-basic variable. A non-negative reduced cost vector indicates optimality.
- **Complementary Slackness Conditions**: Conditions that relate the primal and dual optimal solutions. Specifically, for each constraint, either the constraint is binding (equality holds) or the corresponding dual variable is zero (and vice versa).
- **Sensitivity Analysis**: The study of how changes in the parameters of a linear programming problem (e.g., objective function coefficients, constraint coefficients, or right-hand side values) affect the optimal solution.
- **Basis Matrix** (B): A submatrix of the constraint matrix A formed by the columns corresponding to the basic variables. Its inverse is used in the simplex method to update the solution.
- **Non-basis Matrix** (N): A submatrix of the constraint matrix A formed by the columns corresponding to the non-basic variables.
- **Simplex Tableau**: A tabular representation of the linear programming problem used in the simplex method to track the values of the variables and the objective function during iterations.
- **Dual Simplex Method**: A variant of the simplex method that starts with a dual feasible solution and iteratively moves towards primal feasibility while maintaining dual feasibility, ultimately finding an optimal solution for both the primal and dual problems.
- **Dual Based Phase I Algorithm**: A method used to find an initial feasible solution for the dual problem in linear programming, particularly when the origin is not dual feasible. It involves modifying the dual problem to ensure feasibility at the origin, then applying the dual simplex method.
- **Entering Variable**: In the simplex method, the non-basic variable selected to become a basic variable in the next iteration. The selection is typically based on improving the objective function value.
- **Leaving Variable**: In the simplex method, the basic variable selected to become a non-basic variable in the next iteration. The selection ensures that the resulting solution remains feasible.

- **Elementary Matrix:** A matrix obtained by performing a single elementary row operation (such as swapping two rows, multiplying a row by a non-zero scalar, or adding a multiple of one row to another) on an identity matrix. Used in updating the basis matrix in the simplex method.
- **Convex Set:** A set $S \subseteq \mathbb{R}^m$ such that for any $z_1, z_2 \in S$, $\lambda z_1 + (1 - \lambda)z_2 \in S$ for all $0 \leq \lambda \leq 1$.
- **Convex Combination:** A weighted sum of points $z_1, z_2, \dots, z_n \in \mathbb{R}^m$ of the form $\sum_{i=1}^n \lambda_i z_i$, where $\lambda_i \geq 0$ for all i and $\sum_{i=1}^n \lambda_i = 1$.
- **Convex Hull:** The smallest convex set containing a given set $S \subseteq \mathbb{R}^m$, denoted as $\text{Conv}(S) = \bigcap \{C \mid C \text{ convex}, S \subseteq C\}$.
- **Caratheodory Theorem:** If $z \in \text{Conv}(S)$, then z can be written as a convex combination of at most $m + 1$ points from S .
- **Separation Theorem:** Let P and $\bar{P}$ be two disjoint nonempty polyhedra in $\mathbb{R}^n$. Then there exist disjoint half-spaces H and $\bar{H}$ such that $P \subseteq H$ and $\bar{P} \subseteq \bar{H}$.
- **Farkas' Lemma:** The system $Ax \leq b$ has no solution if and only if there exists a vector $y \geq 0$ such that $y^*TA = 0$ and $y^*Tb < 0$.
- **L^2 Regression (Least Square):** Minimizes the sum of squared errors between observed and predicted values.
- **L^1 Regression:** Minimizes the sum of absolute errors between observed and predicted values.
- **L^∞ Regression:** Minimizes the maximum absolute error between observed and predicted values.
- **Linear Separability:** A dataset is linearly separable if there exists a hyperplane that perfectly separates the data points into two classes.
- **Maximum Margin Classifier:** A classifier that maximizes the minimum distance from the hyperplane to the closest data points.
- **Support Vector Machine (SVM):** A classification algorithm that finds the hyperplane that maximizes the margin between classes.
- **Lagrangian Formulation of SVM:** A method to reformulate the SVM optimization problem using Lagrange multipliers.
- **Geometric Interpretation of SVM:** Understanding SVM optimization through the geometric concept of maximizing the margin between classes.
- **Maximum Inscribed Circle in a Convex Polyhedron:** Finding the largest circle that can fit entirely inside a convex polyhedron.
- **Smallest Ball Problem:** Finding the smallest radius ball that contains all given points in $\mathbb{R}^d$.
- **Monthly Demands:** The predicted sales of ice cream for each month of the year.
- **Average Demand:** The average monthly sales of ice cream over the entire year.
- **Cost of Changing Production:** The cost incurred by the ice cream manufacturer for altering its production levels from one month to the next.
- **Cost of Storage:** The cost incurred by the ice cream manufacturer for storing surplus ice cream from one month to the next.
- **Production Level:** The amount of ice cream produced in a given month.

- **Surplus:** The excess amount of ice cream remaining at the end of a month after meeting the demand.
- **Markowitz Portfolio Selection Model:** A financial model that aims to optimize investment portfolios by maximizing expected return while minimizing risk.
- **Annual Return:** The return on investment for a given asset over a year.
- **Proportion of Investment:** The fraction of a portfolio allocated to a specific asset.
- **Fluctuation:** The deviation of an asset's return from its expected return.
- **Portfolio Variance:** A measure of the risk associated with a portfolio, calculated as the weighted sum of the covariances between all pairs of assets.
- **Covariance:** A measure of the joint variability of two random variables.
- **Smallest Enclosing Ball Problem:** The problem of finding the smallest hypersphere that encloses a given set of points in a d -dimensional space.
- **Maximum Weight Matching:** A combinatorial optimization problem that involves finding a matching in a weighted graph such that the sum of the weights of the edges in the matching is maximized.
- **Machine Scheduling:** A combinatorial optimization problem that involves assigning jobs to machines to minimize the total completion time.
- **Knapsack Problem:** A combinatorial optimization problem that involves selecting a subset of items with weights and values to maximize the total value while not exceeding a given weight capacity.
- **Cutting Stock Problem:** An optimization problem that involves determining how to cut large rolls of material into smaller rolls of various widths to minimize waste.
- **Sparse Solution:** A solution to a system of linear equations that has a limited number of non-zero elements.
- **ℓ_1 Norm:** The sum of the absolute values of the elements of a vector.
- **Fredholm Alternatives:** A statement of the two mutually exclusive possibilities regarding the solvability of the system $Ax = b$.
- **Integer Programming:** A mathematical optimization problem where the decision variables are restricted to integer values.
- **Relaxed Problem:** The linear programming problem obtained by removing the integer constraints from an integer programming problem.
- **Branch-and-Bound:** A method for solving integer programming problems by recursively partitioning the feasible region and bounding the objective function value.
- **Depth-First Search:** A search strategy for exploring the enumeration tree in Branch and Bound, prioritizing deeper branches before wider ones.
- **Pruning:** The process of eliminating branches in the enumeration tree of a Branch and Bound algorithm that cannot lead to a better solution than one already found.
- **Gomory Cuts:** Additional constraints added to a linear programming relaxation of an integer program to cut off non-integer solutions.
- **Network Flow:** A mathematical model representing the flow of commodities through a network, often formulated as a linear program.
- **Node:** A point in a network representing a source, sink, or intermediate point in the flow.

- **Arc:** A directed edge in a network connecting two nodes, representing a flow path with a capacity or cost.
- **Supply:** The amount of commodity available at a node in a network flow problem.
- **Demand:** The amount of commodity required at a node in a network flow problem.
- **Incidence Matrix:** A matrix representing the connections between nodes and arcs in a network, used in the linear programming formulation of network flow problems.
- **Root Node:** A designated node in a network, often used as a reference point for simplifying calculations in network flow problems.
- **Spanning Tree:** A tree that connects all nodes in a network, used in solving network flow problems.
- **Tree Solution:** A solution to a network flow problem where the flow along arcs in a spanning tree is zero.
- **Reduced Cost:** The cost of increasing the flow along a non-basic arc in a network flow problem.
- **Primal Simplex Method:** An algorithm used to solve linear programming problems, applied here to solve network flow problems by iteratively improving a tree solution.
- **Network:** A graph consisting of a set of nodes (N) and a set of arcs (A).
- **Primal Flow:** A flow assignment in a network that satisfies the constraints of the primal problem.
- **Dual Flow:** A flow assignment in a network that satisfies the constraints of the dual problem.
- **Dual Slack:** The difference between the cost of an arc and the difference in dual variables between its endpoints.
- **Two-Phased Network Simplex Method:** A method that first finds a primal feasible solution using artificial costs and then uses the primal network simplex method to find an optimal solution.
- **Parametric Self-Dual Method:** A method that intermingles primal and dual pivots to reduce a perturbation parameter from infinity to zero.
- **Transportation Problem:** A network flow problem where the objective is to minimize the cost of transporting goods from multiple sources to multiple destinations, subject to supply and demand constraints that may be inequalities.
- **Dummy Node:** A node added to a network flow model to handle inequality constraints in supply and demand, allowing for excess supply or unmet demand.
- **Upper Bounded Transshipment Problem:** A network flow problem where the flow along each arc is bounded above by a specified upper bound, in addition to supply and demand constraints.
- **Shortest Path Problem:** A graph problem where the objective is to find the path of minimum cost or distance between a source node and a destination node in a weighted graph.
- **Bellman's Equation:** A recursive equation used to solve the shortest path problem by iteratively computing the shortest distance from each node to a destination node.
- **Dijkstra's Algorithm:** A greedy algorithm for finding the shortest paths from a single source node to all other nodes in a graph with non-negative edge weights.
- **Max Flow:** The maximum amount of flow that can be sent from a source node to a sink node in a network, subject to capacity constraints on the edges.
- **Cut:** A partition of the nodes in a network into two disjoint sets, one containing the source node and the other containing the sink node.

- **Cut Capacity:** The sum of the capacities of all edges that cross a cut from the source set to the sink set.
- **Max-Flow Min-Cut Theorem:** The theorem stating that the maximum flow in a network is equal to the minimum capacity of any cut in the network.
- **Augmenting Path:** A path in a network from the source to the sink along which flow can be increased.
- **Ford-Fulkerson Algorithm:** An algorithm for finding the maximum flow in a network by iteratively finding and augmenting along augmenting paths.
- **Graphical Method:** A method for solving linear programming problems with two variables by visualizing the feasible region and objective function on a graph. The optimal solution is found at a vertex of the feasible region.

2 Problem Types

- **Finding an Initial Feasible Solution:** The Simplex method requires an initial feasible solution. If the origin is not feasible, an auxiliary problem is introduced to find a feasible solution to the original problem.
- **Handling Infeasible Origins:** The origin $(0, 0)$ may not satisfy all constraints in a linear programming problem. Techniques like introducing an auxiliary problem are used to find a feasible starting point.
- **Optimizing in Higher Dimensions:** The Simplex method is extended to linear programming problems with more than two variables, requiring iterative pivoting operations to find the optimal solution.
- **Using Dictionaries to Represent Solutions:** The Simplex method uses dictionaries to represent the current solution and to perform pivot operations efficiently. Each dictionary corresponds to a vertex of the feasible region.
- **Worst-Case Analysis of the Simplex Method:** The Klee-Minty example demonstrates that the Simplex method can require an exponential number of iterations in the worst case, depending on the pivot rule used.
- **Impact of Pivot Rules on Simplex Method Efficiency:** Different pivot rules (e.g., largest-coefficient, largest-increase) significantly affect the number of iterations required by the Simplex method, with some rules exhibiting exponential worst-case behavior while others perform better in practice.
- **Empirical Analysis of Simplex Method Performance:** Empirical studies show that the number of iterations in the Simplex method tends to grow polynomially with the problem size in practice, even though worst-case examples exist.
- **Theoretical Bounds on Simplex Method Iterations:** Theoretical analysis of randomized pivot rules provides bounds on the expected number of iterations, which are better than exponential but not polynomial.
- **Smoothed Complexity of the Simplex Method:** The smoothed complexity analysis suggests that small random perturbations to the input data can significantly improve the performance of the Simplex method, leading to a polynomial number of iterations with high probability.
- **Existence of Polynomial-Time Algorithms for Linear Programming:** Polynomial-time algorithms for linear programming exist, such as interior-point methods, which contrast with the Simplex method's potential for exponential worst-case behavior.
- **Verifying Optimality using Complementary Slackness:** This problem focuses on using the complementary slackness conditions ($x_j z_j = 0$ and $w_i y_i = 0$) to verify whether a given primal and dual feasible solution pair is optimal.

- **Demonstrating the Negative Transpose Property in Simplex Iterations:** This problem involves showing that the negative transpose relationship between the primal and dual tableaux is maintained throughout the simplex method iterations.
- **Proving the Strong Duality Theorem:** This problem involves proving that if the primal problem has an optimal solution, then the dual problem also has an optimal solution, and the optimal objective function values are equal. The proof utilizes the structure of the simplex tableau at optimality and the complementary slackness conditions.
- **Relating Primal and Dual Objective Functions:** This problem involves demonstrating the relationship between the primal and dual objective functions, showing how the primal objective function is always less than or equal to the dual objective function, and proving their equality at optimality.
- **Formulating the Lagrangian for Equality Constraints:** This problem involves constructing the Lagrangian function $L(x, \mu) = f_0(x) - \sum_{j=1}^n \mu_j g_j(x)$ to incorporate equality constraints $g_j(x) = 0$ into the optimization problem.
- **Formulating the Lagrangian for Inequality and Equality Constraints:** This problem involves constructing the Lagrangian function $L(x, \lambda, \mu) = f_0(x) - \sum_{i=1}^m \lambda_i f_i(x) - \sum_{j=1}^n \mu_j g_j(x)$ to incorporate both inequality constraints $f_i(x) \leq 0$ and equality constraints $g_j(x) = 0$ into the optimization problem.
- **Establishing Weak Duality:** This problem involves proving that the optimal value of the primal problem is less than or equal to the optimal value of the dual problem, i.e., $\max_{x \in \mathcal{G}} \{c^T x\} \leq \min_{\lambda \geq 0, \mu} \{L(x^*, \lambda, \mu)\}$.
- **Establishing Strong Duality:** This problem involves proving that under certain conditions (e.g., the primal problem having a finite optimal solution), the optimal values of the primal and dual problems are equal, i.e., $\max_{x \in \mathcal{G}} \{c^T x\} = \min_{\lambda \geq 0, \mu} \{L(x^*, \lambda, \mu)\}$.
- **Deriving the Dual Problem from the Lagrangian:** This problem involves deriving the dual problem by analyzing the maximum of the Lagrangian function with respect to the primal variables, leading to the dual objective function and constraints.
- **Applying Complementary Slackness:** This problem involves using the complementary slackness condition ($x_i^* \lambda_i = 0$ for all i) to verify the optimality of solutions to the primal and dual problems.
- **Formulating the Diet Problem as a Linear Program:** This problem involves translating the description of a diet problem (minimizing cost while meeting nutritional requirements) into a mathematical linear program with an objective function and constraints.
- **Formulating the Dual of the Diet Problem:** This problem involves deriving the dual linear program from the primal linear program representing the diet problem, interpreting the dual variables in the context of the problem.
- **Representing the Diet Problem using Tables:** This problem involves representing the data of the diet problem (nutrient content of foods, minimum nutrient requirements, and food costs) in a tabular format to facilitate the formulation of the linear program.
- **Transforming the Diet Problem into a Pill-Selling Problem:** This problem involves reinterpreting the diet problem as a profit maximization problem by considering the sale of nutrient pills instead of food consumption.
- **Representing Linear Programs in Matrix Notation:** This problem focuses on expressing linear programming problems using matrices and vectors, facilitating the application of the simplex method.
- **Partitioning Variables into Basic and Non-Basic Sets:** This problem involves separating the variables of a linear program into basic and non-basic sets, a crucial step in the simplex algorithm.

- **Expressing the Objective Function and Constraints using Basic and Non-Basic Variables:** This problem deals with rewriting the objective function and constraints of a linear program in terms of basic and non-basic variables, enabling efficient iterative updates in the simplex method.
- **Utilizing Complementary Slackness in Simplex Iterations:** This problem explores the relationship between primal and dual variables (complementary slackness) within the simplex method's iterative process.
- **Solving a Linear Program using the Simplex Method in Matrix Form:** This problem involves applying the simplex method in matrix notation to a specific linear programming problem, demonstrating the iterative process of updating the tableau until optimality is reached.
- **Solving a Linear Program Graphically:** This problem focuses on solving a linear program using a graphical method, identifying the feasible region and the optimal solution point.
- **Deriving the Dual Problem from the Primal Problem:** This problem focuses on deriving the dual linear program from a given primal linear program. The derivation involves techniques such as forming the Lagrangian and manipulating inequalities to obtain the dual objective function and constraints.
- **Handling Unrestricted Variables in Linear Programming:** This problem addresses how to handle linear programs where variables are not restricted to be non-negative. A common approach is to reformulate the problem by splitting each unrestricted variable into the difference of two non-negative variables.
- **Sensitivity Analysis:** Determining how changes in the constraint values affect the optimal solution and objective function value.
- **Solving Linear Programs using the Simplex Method in Matrix Form:** Utilizing matrix operations (e.g., inverse of the basis matrix) to solve linear programs using the simplex method.
- **Deriving the Dual Problem:** Formulating the dual linear program from a given primal linear program.
- **\textbackslash{}textbf\textbackslash{}{}Formulating the Primal and Dual Problems:** \ This problem involves expressing a given optimization problem and its dual counterpart in mathematical notation, including the objective function and constraints. Examples include maximizing or minimizing a linear objective function subject to linear equality or inequality constraints, and then deriving the corresponding dual problem.
- **\textbackslash{}textbf\textbackslash{}{}Solving Linear Programs Using the Dual Simplex Method:** \ This problem focuses on applying the dual simplex method to solve linear programming problems, starting from a dual-feasible but primal-infeasible solution. The method iteratively improves the dual solution while moving towards primal feasibility.
- **\textbackslash{}textbf\textbackslash{}{}Representing and Solving Linear Programs in Matrix Form:** \ This problem involves representing a linear program using matrices and vectors, and then solving it using matrix operations. This includes partitioning variables into basic and non-basic sets, updating the basis matrix, and calculating the optimal solution.
- **\textbackslash{}textbf\textbackslash{}{}Determining Feasibility of Primal and Dual Problems:** \ This problem involves checking whether the origin (all variables equal to zero) is feasible for both the primal and dual problems. If not, a Phase I procedure may be necessary to find a feasible starting point.
- **\textbackslash{}textbf\textbackslash{}{}Proving the equivalence of a convex set and its convex combinations:** \ This problem focuses on demonstrating that a set is convex if and only if it contains all convex combinations of its points. The proof involves showing both directions of the implication.

- **Characterizing the convex hull of a set:** \ This problem involves defining and proving properties of the convex hull of a set. It includes defining the convex hull as the intersection of all convex sets containing the given set and proving that it's equivalent to the set of all convex combinations of finitely many points from the original set.
- **Applying Caratheodory's Theorem to a linear program:** \ This problem uses Caratheodory's Theorem to show that any point in the convex hull of a set in $\mathbb{R}^m$ can be expressed as a convex combination of at most $m + 1$ points. This is demonstrated through the formulation and analysis of a specific linear program.
- **Proving the Separation Theorem for disjoint polyhedra:** \ This problem involves proving that two disjoint nonempty polyhedra in $\mathbb{R}^n$ can be separated by two disjoint half-spaces. The proof utilizes Farkas' Lemma.
- **Proving Farkas' Lemma:** \ This problem focuses on proving Farkas' Lemma, a fundamental result in linear programming, which provides a necessary and sufficient condition for the infeasibility of a system of linear inequalities. The proof uses primal and dual linear programs.
- **Regression:** This problem involves finding parameters that best fit a linear model to a set of data points, minimizing different error functions such as L^2 , L^1 , and L^{infy} norms.
- **Binary Classification:** This problem focuses on finding a hyperplane that separates data points into two classes.
- **Maximum Margin Classifier:** This problem aims to find the hyperplane that maximizes the minimum distance to the closest data points of either class.
- **Support Vector Machine (SVM):** This problem seeks to find the optimal hyperplane that maximizes the margin between two classes, often formulated as a quadratic programming problem.
- **Maximum Inscribed Circle in a Convex Polyhedron:** This problem involves finding the largest circle that can be inscribed within a given convex polyhedron.
- **Smallest Ball Problem:** This problem aims to find the smallest radius ball that encloses a given set of points in a d -dimensional space.
- **Production Planning:** Formulating and solving a linear program to optimize ice cream production over a year, minimizing costs associated with changing production levels and storage, given predicted monthly demands.
- **Workforce Planning:** Determining the optimal mix of regular and temporary employees to minimize labor costs while meeting fluctuating monthly labor demands.
- **Portfolio Selection (Markowitz Model):** Optimizing investment portfolios to maximize expected return while minimizing risk, considering historical data and covariances between assets.
- **Smallest Enclosing Ball Problem:** Finding the smallest ball (hypersphere) that contains a given set of points in a d -dimensional space.
- **Maximum Weight Matching:** Finding a matching in a weighted graph that maximizes the total weight of the selected edges.
- **Machine Scheduling:** Optimizing the scheduling of jobs on multiple machines to minimize the total completion time.
- **Knapsack Problem:** Selecting a subset of items with weights and values to maximize the total value while not exceeding a weight capacity.

- **Cutting Stock Problem:** Determining the optimal way to cut large rolls of material into smaller rolls of various widths to minimize waste.
- **Sparse Solutions of Linear Systems:** Finding a solution to an underdetermined system of linear equations that has a limited number of non-zero elements.
- **Determining the existence of a non-negative solution to $Ax = b$:** This problem investigates whether there exists a vector x with non-negative components that satisfies the equation $Ax = b$. Several equivalent conditions are presented using a vector y .
- **Determining the existence of a solution to $Ax \leq b$:** This problem explores whether a solution x exists that satisfies the inequality $Ax \leq b$. Different conditions are presented depending on whether x is restricted to be non-negative.
- **Characterizing the conditions for the non-existence of a solution to $Ax \leq b$:** This problem focuses on determining the conditions under which no solution x exists for the inequality $Ax \leq b$. The conditions are expressed in terms of the existence of a vector y satisfying specific constraints.
- **Integer Programming:** A linear programming problem where some or all variables are restricted to integer values.
- **Solving Integer Programs using Branch and Bound:** A method for solving integer programs by recursively partitioning the feasible region and solving relaxed linear programs.
- **Applying Gomory Cuts:** A method to generate additional constraints (cuts) to eliminate non-integer solutions from the feasible region of an integer program.
- **Formulating Network Flow Problems as Linear Programs:** This problem involves representing a network flow problem, characterized by nodes, arcs, supply/demand at nodes, and arc costs, as a linear program with an objective function minimizing total transportation cost and constraints ensuring flow conservation at each node.
- **Analyzing the Incidence Matrix of a Network:** This problem focuses on understanding the properties of the incidence matrix, including its rank and the relationship between its submatrices and spanning trees in the network. The goal is to understand how the structure of the incidence matrix relates to the solvability and solution methods for the network flow problem.
- **Relating Spanning Trees to Bases in Network Flow Problems:** This problem explores the connection between spanning trees in a network and bases in the simplex method. The goal is to understand how spanning trees can be used to represent basic feasible solutions and how this relationship can be exploited to solve network flow problems efficiently.
- **Applying the Primal Simplex Method to Network Flow Problems:** This problem involves using the primal simplex method to iteratively improve a feasible solution to a network flow problem, represented by a spanning tree, until an optimal solution is reached. The method involves identifying entering and leaving variables, updating the spanning tree, and updating primal and dual variables.
- **Finding a primal feasible flow in a network:** This problem focuses on finding a flow vector x such that $\sum_j x_{ij} = b_i$ and $x_{ij} \geq 0$ for $(i, j) \in T$, where T is a spanning tree. The solution might have negative components, requiring further adjustments.
- **Selecting entering and leaving arcs in the primal network simplex method:** This problem focuses on choosing an entering arc with $z_{ij} < 0$ and a leaving arc to form a cycle, updating primal and dual flows to improve the solution.
- **Selecting the entering arc in the dual network simplex method:** This problem involves choosing an entering arc that bridges two subtrees in the opposite direction from the leaving arc, and has the smallest dual slack among all such arcs.

- **Updating primal and dual flows after selecting entering and leaving arcs:** This problem involves updating the primal and dual flows along the cycle formed by the entering and leaving arcs, ensuring that the spanning tree remains valid and the flow values are adjusted correctly.
- **Updating dual variables in the primal network simplex method:** This problem focuses on updating dual variables after a primal pivot step, considering the cases where the entering arc crosses between trees containing and not containing the root node.
- **Finding an initial feasible solution using a two-phased network simplex method:** This problem involves modifying costs to ensure dual feasibility of an initial tree solution, then using the primal network simplex method to find an optimal solution.
- **Finding an initial feasible solution using the primal network simplex method:** This problem involves modifying supplies to ensure primal feasibility of an initial tree solution, then using the dual network simplex method to find an optimal solution.
- **Solving network flow problems using a parametric self-dual method:** This problem involves interweaving primal and dual pivots to reduce a perturbation parameter μ from ∞ to zero.
- **\textbackslash{}textbf\textbackslash{}{}Formulating Transportation Problems with Inequality Constraints:** \ This problem involves modeling a transportation network where supply at sources may exceed demand at destinations, or vice versa. This requires the introduction of slack variables or dummy nodes to balance the network flow equations.
- **\textbackslash{}textbf\textbackslash{}{}Formulating and Solving Upper Bounded Transshipment Problems:** \ This problem type extends the standard network flow problem by adding upper bounds on the flow along each arc. This requires modifications to the standard linear programming formulation, often involving the introduction of slack variables to handle the upper bounds.
- **\textbackslash{}textbf\textbackslash{}{}Formulating and Solving Shortest Path Problems as Network Flows:** \ This problem involves finding the shortest path between two nodes in a network by formulating it as a linear program. The constraints ensure that exactly one unit of flow leaves the source, exactly one unit enters the destination, and flow is conserved at intermediate nodes.
- **\textbackslash{}textbf\textbackslash{}{}Solving Shortest Path Problems using Bellman's Equation:** \ This problem focuses on finding the shortest path using dynamic programming techniques based on Bellman's equation. The algorithm iteratively updates distance estimates until convergence.
- **\textbackslash{}textbf\textbackslash{}{}Solving Shortest Path Problems using Dijkstra's Algorithm:** \ This problem involves finding the shortest path using Dijkstra's algorithm, which efficiently explores the network by iteratively selecting the node with the smallest current shortest distance estimate.
- **Maximum Flow Problem:** Finding the maximum flow from a source node to a sink node in a network, subject to capacity constraints on the edges.
- **Max-Flow Min-Cut Theorem:** Proving that the maximum flow in a network is equal to the minimum capacity of any cut separating the source and sink nodes.
- **Implementing the Ford-Fulkerson Algorithm:** Describing and implementing the steps of the Ford-Fulkerson algorithm, which iteratively finds augmenting paths to increase the flow until the maximum flow is reached.
- **Linear Programming Formulation of Maximum Flow:** Formulating the maximum flow problem as a linear program, including the objective function and constraints.
- **Solving Linear Programs Graphically:** A geometric method for solving linear programs with two variables is presented, involving identifying the feasible region and finding the optimal solution at a vertex of the feasible region.

- **Formulating the Dual Problem:** Given a primal linear program, the corresponding dual linear program is formulated, showing the transformation of coefficients and constraints.
- **Finding Optimal Solutions Using Duality:** The optimal solutions to both the primal and dual problems are found and verified using complementary slackness conditions.
- **Geometric Interpretation of Dual Constraints:** The dual constraints are interpreted geometrically as half-spaces in the primal variable space, and the optimal dual solution is identified as the intersection of these half-spaces.
- **Performing Sensitivity Analysis:** The impact of changes in the objective function coefficients (c) and constraint values (b) on the optimal solution is analyzed using formulas involving the optimal primal and dual solutions.
- **Determining the Range of Parameter Changes Maintaining Optimality:** Given an optimal solution to a linear program, the range of values for a parameter in either the objective function or constraints is determined such that the current optimal solution remains optimal.
- **Representing Linear Programs in Matrix Form:** A linear program is represented in matrix notation, separating variables into basic and non-basic sets, and expressing the objective function and constraints using these sets.
- **Applying Sensitivity Analysis to a Specific Problem:** Sensitivity analysis is applied to a specific linear programming problem to determine the impact of changes in the constraint values on the optimal solution value.
- **Formulating and Solving Primal and Dual Problems:** A primal linear programming problem is presented, and its corresponding dual problem is formulated and solved.
- **Analyzing Complementary Slackness Conditions:** The complementary slackness conditions are analyzed to determine the optimal solution and the range of parameter values that maintain optimality.

3 Algorithm Solutions

- **Simplex Method:** Iteratively move from one vertex of the feasible region to another by setting non-basic variables to zero and choosing a new set of (non)basic variables to improve the objective function value. This is equivalent to moving from vertex to vertex in the feasible region.
- **Auxiliary Problem:** When the origin is not feasible, introduce a new variable x_0 and modify the constraints to create an auxiliary problem that maximizes $-x_0$. Solve this auxiliary problem using the Simplex method. If the optimal solution has $x_0 = 0$, then the solution is feasible for the original problem.
- **Largest-Coefficient Rule:** Choose the variable with the largest coefficient in the objective function row to enter the basis.
- **Largest-Increase Rule:** Choose the variable whose entrance into the basis brings about the largest increase in the objective function.
- **Bland's Rule:** Choose the variable with the smallest index among those eligible to enter the basis.
- **Randomized Pivot Rule:** Randomly permute the indices of the variables; then use Bland's rule for entering and the lexicographic method for leaving variables.
- **Weak Duality:** $c^T x \leq b^T y$
- **Strong Duality:** If (P) has an optimal solution x^* , then (D) has an optimal solution y^* , and $c^T x^* = b^T y^*$.
- **Complementary Slackness:** Primal feasible x and dual feasible y are optimal if and only if $x_j z_j = 0$ for all j and $w_i y_i = 0$ for all i .

- **Lagrangian Function:** $L(x, \lambda, \mu) = f_0(x) - \sum_{i=1}^m \lambda_i f_i(x) - \sum_{j=1}^n \mu_j g_j(x)$
- **Primal Problem (Diet Problem):** $\text{Minimize } Z(x) = \sum_j c_j x_j \text{ subject to } \sum_j a_{ij} x_j \leq b_i, x_j \geq 0$
- **Dual Problem (Diet Problem):** $\text{Maximize } Z(y) = \sum_i b_i y_i \text{ subject to } \sum_j a_{ij} y_j \leq c_j, y_i \geq 0$
- **Simplex Method in Matrix Form:** The simplex method iteratively improves a feasible solution by exchanging basic and non-basic variables until an optimal solution is found. This involves updating the objective function and constraints using matrix operations, specifically involving the basis matrix B and its inverse B^{-1} , and the non-basic variable matrix N . The objective function is expressed as $Z = c_B^T B^{-1} b + (c_N^T - c_B^T B^{-1} N) x_N$, where c_B and c_N are the cost vectors for basic and non-basic variables, respectively, and b is the right-hand side vector of the constraints.
- **Dual Problem Derivation using Lagrangian Multipliers:** The dual problem is derived from the primal problem by constructing the Lagrangian function, $L(x, y, z) = c^T x - y^T (Ax - b) - z^T (x - u)$, where y and z are Lagrange multipliers. By taking the partial derivative with respect to x and setting it to zero, and considering the constraints $y \geq 0$ and $z \geq 0$, the dual problem is obtained.
- **Reformulation of Primal Problem with Unrestricted Variables:** A primal problem with unrestricted variables x can be reformulated by splitting each variable into its positive and negative parts, $x = x^+ - x^-$, where $x^+ \geq 0$ and $x^- \geq 0$. This allows the application of standard linear programming techniques.
- **Sensitivity Analysis:** This technique analyzes how changes in the parameters of a linear programming problem (e.g., objective function coefficients, constraint coefficients, or right-hand side values) affect the optimal solution and objective function value. It often involves calculating the range of parameter changes that maintain the current optimal basis.
- **Shadow Price:** The shadow price of a constraint is the rate of change in the optimal objective function value for a unit change in the right-hand side of that constraint. It is given by the corresponding optimal dual variable.
- **Caratheodory's Theorem:** If $z \in \text{Conv}(S)$, then z can be written as a convex combination of at most $m + 1$ points from S .
- **Farkas' Lemma:** The system $Ax \leq b$ has no solution if and only if there exists a vector $y \geq 0$ such that $y^T A = 0$ and $y^T b < 0$.
- **Separation Theorem:** Let P and $\bar{P}$ be two disjoint nonempty polyhedra in $\mathbb{R}^n$. Then there exist disjoint half-spaces H and $\bar{H}$ such that $P \subset H$ and $\bar{P} \subset \bar{H}$.
- **$\mathbb{L}^2$ Regression (Least Square):** $\min_{x_1, \dots, x_n} \sum_{i=1}^m \left| (b_i - \sum_{j=1}^n a_{ij} x_j) \right|^2$ or $\min_{\|b - AX\|_2} \|b - AX\|_2^2$, solved by the normal equation $A^T A \hat{X} = A^T b$, yielding $\hat{X} = (A^T A)^{-1} A^T b$ if $(A^T A)^{-1}$ exists.
- **$\mathbb{L}^1$ Regression:** $\min_{x_1, \dots, x_n} \sum_{i=1}^m |b_i - \sum_{j=1}^n a_{ij} x_j|$ or $\min_X \|b - AX\|_1$, solved by introducing slack variables $\hat{\epsilon}_i$ such that $-\hat{\epsilon}_i \leq b_i - \sum_{j=1}^n a_{ij} x_j \leq \hat{\epsilon}_i$ and minimizing $\sum_{i=1}^m \hat{\epsilon}_i$.
- **$\mathbb{L}^\infty$ Regression:** $\min_{x_1, \dots, x_n} \max_{i=1, \dots, m} |b_i - \sum_{j=1}^n a_{ij} x_j|$ or $\min_X \|b - AX\|_\infty$, solved by introducing a slack variable δ such that $-\delta \leq b_i - \sum_{j=1}^n a_{ij} x_j \leq \delta$ for all i and minimizing δ .

- **Lagrangian Formulation of SVM:** $\min \alpha \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^N \alpha_i$
subject to $\sum_{i=1}^N \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C, \forall i$.
- **Distance of a Point from a Hyperplane:** $d_i = \frac{|a_1 s_i + a_2 s_i^2 - b|}{\sqrt{a_1^2 + a_2^2}}$
- **Ice Cream Production Planning:** $\min 50 \sum_{i=1}^{12} (Y_i + Z_i) + 20 \sum_{i=1}^{12} S_i$ s.t. $X_i + S_{i-1} - S_i = d_i$,
 $X_i - X_{i-1} = Y_i - Z_i$, $X_0 = 0$, $S_0 = 0$, $S_{12} = 0$, $X_i, Y_i, Z_i, S_i \geq 0$
- **Workforce Planning Reformulation:** $\min 25 \sum_{i=1}^{12} Y_i + (17.5 \text{ times } 12) X$ s.t. $a_i - X \leq Y_i$, $0 \leq Y_i$,
 $X \geq 0$
- **Smallest Enclosing Ball:** minimize $x^* T Q^T Q x - \sum_{j=1}^n x_j p_j^T p_j$ subject to $\sum_{j=1}^n x_j = 1$, $x_j \geq 0$
- **Maximum Weight Matching:** $\max \sum_{e \in E} w_e x_e$ s.t. $\sum_{e \in \delta(v)} x_e = 1$ for all vertices v ,
 $x_e \in \{0, 1\}$ for all edges e
- **Machine Scheduling:** $\min \pi_i$ s.t. $\sum_{i \in M} x_{ij} = 1$ for $j \in J$, $\sum_{j \in J} d_{ij} x_{ij} \leq p_i$ for $i \in M$,
 $x_{ij} \in \{0, 1\}$ for all i, j
- **0-1 Knapsack Problem:** maximize $\sum_{j=1}^n c_j x_j$ subject to $\sum_{j=1}^n a_j x_j \leq b$, $x_j \in \{0, 1\}$, $j = 1, 2, \dots, n$
- **Cutting Stock Problem:** $\min \sum_{i=1}^{12} x_i$ subject to constraints based on cutting patterns P_i and demand for each roll width
- **Sparse Solutions of Linear System:** $\min \|x\|_1$ s.t. $Ax = b$
- **Branch and Bound:** This algorithm solves integer programming problems by recursively partitioning the feasible region into subproblems, solving the relaxed linear program for each subproblem, and using the objective function values to prune branches that cannot lead to a better integer solution.
- **Gomory Cuts:** This algorithm adds cutting planes (constraints) to the relaxed linear program to eliminate non-integer solutions. The cuts are derived from the fractional parts of the coefficients in the optimal dictionary of the relaxed problem. The algorithm iteratively adds cuts until an integer solution is found.
- **Network Flow Problem Formulation:** The network flow problem is formulated as a linear program with the objective function $\min \sum_{(i,j) \in A} c_{ij} x_{ij}$ subject to flow conservation constraints $\sum_{i \in X} x_{ik} - \sum_{j \in X} x_{kj} = -b_k$ at each node k and non-negativity constraints $x_{ij} \geq 0$.
- **Primal Simplex Method for Network Flows:** The primal simplex method is adapted for network flows by iteratively selecting an entering variable (arc with the most negative reduced cost), identifying a leaving variable (arc in the cycle formed by the entering variable and the spanning tree), updating the spanning tree, and updating primal and dual variables until all dual slacks are non-negative.
- **Finding a Spanning Tree Basis:** A square submatrix of the modified incidence matrix $\tilde{A}$ (incidence matrix with a root node row removed) is a basis if and only if the arcs corresponding to its columns form a spanning tree.
- **Dual Variable Calculation from Spanning Tree:** Given a spanning tree, dual variables are computed iteratively starting from a root node using the equation $y_j - y_i = c_{ij}$ for arcs (i, j) in the spanning tree. Dual slacks for arcs not in the spanning tree are then computed using $z_{ij} = y_j - y_i + c_{ij}$.
- **Parametric Self-Dual Network Simplex Method:** This method interweaves primal and dual pivots to reduce a perturbation parameter μ from ∞ to 0, finding an optimal solution by simultaneously improving primal and dual feasibility.
- **Bellman-Ford Algorithm:** $v_i = \min_{j \in V} \{c_{ij} + v_j\}$, $v_r = 0$

- **Ford-Fulkerson Algorithm (Augmented Path):** This algorithm iteratively finds augmenting paths in a network flow graph and updates the flow along these paths to increase the total flow from the source to the sink until no more augmenting paths can be found. The algorithm terminates when all labeled nodes are scanned.
- **Max-Flow Min-Cut Theorem:** The maximum flow from the source to the sink in a network is equal to the minimum capacity of any cut in the network.
- **Graphical Method:** Solve a linear program with two variables by plotting the constraints to define the feasible region and then finding the vertex that maximizes (or minimizes) the objective function.
- **Dual Problem Derivation:** Given a primal problem $\text{max } c^T x \text{ s.t. } Ax \leq b, x \geq 0$, the dual problem is $\text{min } b^T y \text{ s.t. } A^T y \geq c, y \geq 0$.